241080076 Vishwajeet Waghmare | 241080077 Awwab Wadekar
241080078 Yash Wadhwa | 241080082 Abiraj Kangotra

Hybrid Offline-First HMS: Project Estimation

2B. Effort Estimation for Project

Key Project Parameters

- **Team Scale:** 4 Student Developers
- **Target Deadline:** 14 Weeks (3.5 Months Max)
- **Weekly Team Bandwidth:** ~60 to 72 Person-Hours/Week (~15–18 hrs/week per student)
- **Software Size:** 97 Function Points | 5.0 KLOC (5,000 Lines of Code)

1. Function Point (FP) Based Estimation

Function Point Analysis evaluates software size by measuring the functional requirements delivered to the end-user.

A. Unadjusted Function Point (UFP) Breakdown

Function Type	Description	Count	Weight (Avg)	Total FP
External Inputs (EI)	**Patient Intake, Doctor Schedule Update, Bed Matrix Toggle, Stock Entry**	**4**	**4**	**16**
External Outputs (EO)	**Patient Token Display, Low-Stock Alert, Queue**	**3**	**5**	**15**

	Broadcast Packet			
External Inquiries (EQ)	**Doctor Availability Query, Bed Matrix Status, Patient Search**	**3**	**4**	**12**
Internal Logical Files (ILF)	**Patient Table, Doctor/Schedule Table, Bed Status Table, Stock Ledger**	**4**	**10**	**40**
External Interface Files (EIF)	**Cloud REST API Interface, Local LAN Socket Broadcast Interface**	**2**	**7**	**14**
Total Unadjusted FPs				**97 FP**

B. Value Adjustment Factor (VAF) & Final FP

Assuming a Degree of Influence ($\sum C_i = 35$) across system characteristics (high emphasis on offline data integrity, LAN communication, and UI response speed):

$$VAF = 0.65 + (0.01 \times 35) = 1.00$$
$$\text{Final FP} = 97 \times 1.00 = \mathbf{97}\ \text{FPs}$$

C. Effort & Schedule Derivation

With modern high-level languages (Python, C#, FastAPI), average developer productivity is 20 FP per Person-Month (PM) (1 PM = 152 working hours).

$$\text{Effort (FP)} = \frac{97\ \text{FP}}{20\ \text{FP / PM}} = \mathbf{4.85}\ \text{Person-Months (737 Person-Hours)}$$

$$\text{Timeline (at 18 hrs/wk/dev)} = \frac{737\ \text{Hours}}{4\ \text{Devs} \times 18\ \text{hrs/wk}} \approx \mathbf{10.2}\ \text{Weeks}$$

2. Lines of Code (LOC) Based Estimation

With 4 developers, module allocation directly aligns with individual software components, removing code ownership bottlenecks:

A. Module-Wise 4-Developer Allocation

Developer Role	Module / Subsystem Responsibility	Language / Stack	Estimated LOC
Dev 1 (Desktop UI Lead)	**Receptionist & Doctor Desktop GUI Views & Keyboard UX**	**Python (PyQt) / C#**	**1,500**
Dev 2 (Local Engine Lead)	**SQLite Schema, Local Cache, Transaction Logs & Validation**	**Python / C#**	**1,100**

Dev 3 (Embedded Lead)	**Raspberry Pi / ESP32 Firmware, HDMI Display & Queue UI**	**MicroPython / C++**	**1,000**
Dev 4 (Cloud Sync Lead)	**FastAPI Cloud REST API, Background Sync Worker, DB Sync**	**Python (FastAPI)**	**1,400**
Total Codebase Size			**5,000 LOC (5.0 KLOC)**

B. Effort & Schedule Derivation

At a standard student productivity rate of 1.0 KLOC (1,000 Lines of Code) per Person-Month:

$$\text{Effort (LOC)} = \frac{5.0 \text{ KLOC}}{1.0 \text{ KLOC / PM}} = \mathbf{5.00} \text{ Person-Months } (\approx 760 \text{ Person-Hours})$$

$$\text{Timeline (at 18 hrs/wk/dev)} = \frac{760 \text{ Hours}}{4 \text{ Devs} \times 18 \text{ hrs/wk}} \approx \mathbf{10.5} \text{ Weeks}$$

3. COCOMO Model & Communication Overhead

We apply the Basic & Intermediate COCOMO Model under Organic Mode ($a_b = 2.4, b_b = 1.05$ **).**

A. Nominal Effort Calculation

Filtering out non-algorithmic boilerplate UI/Markup leaves 2.1 KLOC of core system logic:

$$\text{Nominal Effort } (E_{\text{nom}}) = 2.4 \times (2.1)^{1.05} = 2.4 \times 2.181 \approx \mathbf{5.23} \text{ Person-Months } (\approx 795 \text{ Hours})$$

B. Communication Overhead Factor

Adding a 4th developer increases communication channels from 3 to 6:

$$\text{Communication Pathways} = \frac{N(N-1)}{2} = \frac{4(3)}{2} = \mathbf{6}\ \text{Channels}$$

To account for integration overhead across 6 communication pathways, an intermediate Effort Adjustment Factor (EAF = 1.05) is applied:

$$\text{Adjusted Effort } (E_{\text{adj}}) = 5.23 \times 1.05 = \mathbf{5.49}\ \text{Person-Months } (\ 834\ \text{Person-Hours})$$

C. Full-Time Equivalence (FTE) & Safety Buffer

COCOMO calculates a required full-time staffing level of 1.12 Full-Time Engineers:

$$\text{4-Student Part-Time FTE Capacity} = 4\ \text{Devs} \times \left(\frac{15\ \text{hrs/wk}}{40\ \text{hrs/wk}}\right) = \mathbf{1.50}\ \text{FTE}$$

Capacity Buffer: The 4-developer setup provides a +33% safety buffer (1.50 FTE vs. 1.12 FTE required), allowing the team to comfortably absorb academic exams, network drop testing, and sync conflict resolution without project delays.

4. Method Reconciliation & Summary Comparison

Estimation Method	Input Metric	Effort (PM / Hours)	14-Week Weekly Load	Duration at 18 hrs/wk
Function Point (FP)	**97 FPs**	**4.85 PM (~737 Hrs)**	**~13.1 Hrs/wk/ dev**	**10.2 Weeks**
Lines of Code (LOC)	**5.0 KLOC**	**5.00 PM (~760 Hrs)**	**~13.5 Hrs/wk/ dev**	**10.5 Weeks**

COCOMO (Adjusted)	**2.1 Core KLOC**	**5.49 PM (~834 Hrs)**	**~14.8 Hrs/wk/ dev**	**11.5 Weeks**
Target 4-Dev Plan	**4 Develo pers**	**~5.00 PM (~760 Hrs)**	**~13.5 Hrs/wk/ dev**	**10.5 Wks (+3.5 Wk Buffer)**

Conclusion: All three software engineering estimation methodologies converge between **4.85 and 5.23 Person-Months (~737 to 795 Person-Hours)**. This confirms that the proposed scope is mathematically realistic and technically achievable for a 4-student team operating over a 3.5-month project lifecycle. We have successfully verified our approach via the LOC, FP and COCOMO estimation methods.